

run the second parser on the same input. We said that consuming at least one character should result in a committed parse, and that `attempt(p)` converts committed failures of `p` to uncommitted failures.

We can support the behavior we want by adding one more piece of information to the Failure case of Result—a Boolean value indicating whether the parser failed in a committed state:

```
case class Failure(get: ParseError,
                  isCommitted: Boolean) extends Result[Nothing]
```

The implementation of `attempt` just cancels the commitment of any failures that occur. It uses a helper function, `uncommit`, which we can define on `Result`:

```
def attempt[A](p: Parser[A]): Parser[A] =
  s => p(s).uncommit

def uncommit: Result[A] = this match {
  case Failure(e, true) => Failure(e, false)
  case _ => this
}
```

Now the implementation of `or` can simply check the `isCommitted` flag before running the second parser. In the parser `x or y`, if `x` succeeds, then the whole thing succeeds. If `x` fails in a committed state, we fail early and skip running `y`. Otherwise, if `x` fails in an uncommitted state, we run `y` and ignore the result of `x`:

```
def or[A](x: Parser[A], y: => Parser[A]): Parser[A] =
  s => x(s) match {
    case Failure(e, false) => y(s)
    case r => r
  }
```

← Committed failure or success skips running y.

9.6.5 Context-sensitive parsing

Now for the final primitive in our list, `flatMap`. Recall that `flatMap` enables context-sensitive parsers by allowing the selection of a second parser to depend on the result of the first parser. The implementation is simple—we advance the location before calling the second parser. Again we use a helper function, `advanceBy`, on `Location`. There is one subtlety—if the first parser consumes any characters, we ensure that the second parser is committed, using a helper function, `addCommit`, on `ParseError`.

Listing 9.3 Using `addCommit` to make sure our parser is committed

```
def flatMap[A,B](f: Parser[A])(g: A => Parser[B]): Parser[B] =
  s => f(s) match {
    case Success(a, n) => g(a)(s.advanceBy(n))
                          .addCommit(n != 0)
                          .advanceSuccess(n)
    case e@Failure(_, _) => e
  }
```

Commit if the first parser has consumed any characters.

Advance the source location before calling the second parser.

If successful, we increment the number of characters consumed by `n`, to account for characters already consumed by `f`.

`advanceBy` has the obvious implementation. We simply increment the offset:

```
def advanceBy(n: Int): Location =
  copy(offset = offset+n)
```

Likewise, `addCommit`, defined on `ParseError`, is straightforward:

```
def addCommit(isCommitted: Boolean): Result[A] = this match {
  case Failure(e,c) => Failure(e, c || isCommitted)
  case _ => this
}
```

And finally, `advanceSuccess` increments the number of consumed characters of a successful result. We want the total number of characters consumed by `flatMap` to be the sum of the consumed characters of the parser `f` and the parser produced by `g`. We use `advanceSuccess` on the result of `g` to ensure this:

```
def advanceSuccess(n: Int): Result[A] = this match {
  case Success(a,m) => Success(a,n+m)
  case _ => this
}
```

← If unsuccessful, leave
the result alone.



EXERCISE 9.15

Implement the rest of the primitives, including `run`, using this representation of `Parser`, and try running your JSON parser on various inputs.¹⁹



EXERCISE 9.16

Come up with a nice way of formatting a `ParseError` for human consumption. There are a lot of choices to make, but a key insight is that we typically want to combine or group labels attached to the same location when presenting the error as a `String` for display.



EXERCISE 9.17

Hard: The `slice` combinator is still less efficient than it could be. For instance, `many(char('a')).slice` will still build up a `List[Char]`, only to discard it. Can you think of a way of modifying the `Parser` representation to make slicing more efficient?

¹⁹ You'll find, unfortunately, that it causes stack overflow for large inputs (for instance, `[1,2,3,...10000]`). One simple solution to this is to provide a specialized implementation of `many` that avoids using a stack frame for each element of the list being built up. So long as any combinators that do repetition are defined in terms of `many` (which they all can be), this solves the problem. See the answers for discussion of more general approaches.